MODULE 11

Policy Authorization & Security Hardening

Resource ownership (CourseInstructorHandler) · Angular roleGuard & jwtInterceptor · Rate limiting · Security headers

role.guard.ts

```
export const roleGuard = (allowedRoles:
string[]): CanActivateFn => {

  return (route, state) => {

    const authService =
inject(AuthService);

    if
(authService.hasAnyRole(allowedRoles))
return true;

    return
router.createUrlTree(['/unauthorized']);
  };
};
```

1. Resource-Based Authorization: Beyond Simple Roles

1. Resource-Based Authorization: Beyond Simple Roles

- Role checks ([Authorize(Roles = "Instructor")]) are coarse. Instructors should only be allowed to modify courses they own (InstructorId == UserId).

```
// CourseInstructorHandler.cs

public class CourseInstructorHandler : AuthorizationHandler<CourseInstructorRequirement, Course> {

    protected override Task HandleRequirementAsync(AuthorizationHandlerContext context, CourseInstructorRequirement requirement, Course
resource) {

        var userId = context.User.FindFirstValue(ClaimTypes.NameIdentifier);
        var isInstructor = context.User.IsInRole("Instructor");
        var isAdmin = context.User.IsInRole("Admin");

        if (isAdmin || (isInstructor && resource.InstructorId == userId)) context.Succeed(requirement);
        return Task.CompletedTask;
    }
}


// CourseController.cs
[HttpPut("{id}")]
public async Task<IActionResult> UpdateCourse(int id, UpdateCourseDto dto) {

    var course = await _context.Courses.FindAsync(id);
    if (course == null) return NotFound();

    var authResult = await _authService.AuthorizeAsync(User, course, "CanEditCourse");
```

2. Angular SPA Defense: Guards, Interceptors & Structural UI Hiding

- Client-side security provides responsive UX (never a replacement for server API authorization).

Angular Security & Template Directives

```
// role.guard.ts
export const roleGuard = (allowedRoles: string[]): CanActivateFn => {
    return (route, state) => {
        const authService = inject(AuthService);
        const router = inject(Router);
        if (authService.hasAnyRole(allowedRoles)) return true;
        return router.createUrlTree(['/unauthorized']);
    };
};


// jwt.interceptor.ts
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
    const token = inject(AuthService).getToken();
    if (token) req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
    return next(req);
};


<!-- Structural UI hiding in Angular template -->
@if (authService.hasRole('Admin') || authService.isCourseOwner(course.instructorId)) {

  <button (click)="openEditModal(course.id)" class="btn btn-primary">Edit Course</button>
}
```

3. API Security Hardening: Rate Limiting & Security Headers

Program.cs

```
// Program.cs - Rate Limiter (Fixed Window 5 attempts / min on login)

builder.Services.AddRateLimiter(options => {

    options.AddFixedWindowLimiter("LoginPolicy", opt => {

        opt.PermitLimit = 5;
        opt.Window = TimeSpan.FromMinutes(1);
        opt.QueueLimit = 0;
    });

    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
});


// Security Headers Middleware
app.Use(async (context, next) => {

    context.Response.Headers.Append("X-Frame-Options", "DENY");

    context.Response.Headers.Append("X-Content-Type-Options", "nosniff");

    context.Response.Headers.Append("Content-Security-Policy", "default-src 'self'");

    await next();
});
```

4. Session 3 Traps & Common Mistakes

Mistake	Symptom	Enterprise Fix
UX-Only Security	UI button hidden, but API PUT endpoint unprotected	Always back Angular @if with API AuthorizeAsync or [Authorize]
Throwing Null Ref in Handlers	API returns 500 Internal Server Error on unauthorized request	Gracefully return Task.CompletedTask without calling context.Succeed()
Missing Rate Limit Policy	Brute force login attacks overload server	Attach [EnableRateLimiting("LoginPolicy")] to POST /api/auth/login

M11 Complete: Security & Authentication

Full security architecture sprint complete for the TMS platform.